Application Management Current-State Assessment & Modernization Roadmap (Microsoft Endpoint / Intune)

Implementation Solution Guide

Version 1.0.0 · 2026-07-20

Contents

Executive Summary — Application Management Current-State Assessment & Modernization Roadmap 4

What you get . . . 4

What this delivers . . . 4

The standard deployment model . . . 5

Outcome . . . 5

1. Solution Overview — Application Management Current-State Assessment & Modernization Roadmap 5

What this solution is . . . 5

What this solution is NOT . . . 6

The maturity model — `APPASMT – Maturity Model (9 Dimensions x 5 Levels)` . . . 6

How the assessment runs . . . 8

Everything is read-only . . . 8

Named artifacts produced . . . 8

Benefits . . . 9

Limitations — be honest with the buyer . . . 9

The standard deployment model . . . 10

2. Requirements Guide — Application Management Current-State Assessment & Modernization Roadmap 10

2.1 Least-privilege read-only tenant access (Intune role: Read Only Operator) . . . 10

2.2 Microsoft Graph PowerShell SDK with read-only scopes consented . . . 11

2.3 An agreed Assessment Charter (scope, dimensions, target levels) . . . 11

2.4 Named stakeholders available for interview . . . 12

2.5 Documentary artifacts requested and a place to store the Evidence Pack . . . 12

2.6 A workstation that meets the collection prerequisites . . . 12

2.7 Government (GCC High / DoD) — confirm the sovereign-cloud endpoints . . . 13

Requirements checklist . . . 13

Global Requirements — Intune Solutions (category-level) 13

G1. Microsoft Entra ID tenant . . . 14

G2. Microsoft Intune subscription . . . 14

G3. Administrative roles . . . 14

G4. Device enrollment in Intune . . . 14

G5. Pilot security group . . . 14

G6. Supported Windows baseline . . . 14

G7. Network access to Microsoft cloud endpoints . . . 14

3. Implementation Guide — Application Management Current-State Assessment & Modernization Roadmap 15
Step 1 — Charter the assessment (scope & targets) . . . 15
Step 2 — Obtain and verify read-only access (Access) . . . 15
Step 3 — Request and collect documentary artifacts (Collect) . . . 16
Step 4 — Run the structured interviews (Collect) . . . 16
Step 5 — Collect the technical evidence (Collect) . . . 16
Step 6 — Score each dimension against the evidence tables (Score) . . . 17
Step 7 — Turn gaps into findings (Findings) . . . 21
Step 8 — Sequence the roadmap and run the readout (Roadmap → Readout) . . . 21
5. Testing Guide — Application Management Current-State Assessment & Modernization Roadmap 22
Test 1 — Access is genuinely read-only . . . 22
Test 2 — Every dimension has evidence, or an explicit gap . . . 23
Test 3 — Every score cites its evidence . . . 23
Test 4 — The score matches the evidence table (repeatability spot-check) . . . 23
Test 5 — Interview claims and technical data are reconciled . . . 23
Test 6 — Every gap becomes a finding . . . 23
Test 7 — Every finding lands in exactly one wave, with dependencies drawn . . . 24
Test 8 — The executive summary contains all five mandatory elements . . . 24
Test 9 — The assessment is repeatable end to end . . . 24
Pass/fail gate . . . 24
6. Troubleshooting Guide — Application Management Current-State Assessment & Modernization Roadmap 24
Access and connection . . . 25
Data quality and scoring . . . 25
Stakeholders and scope . . . 26
Reporting integrity . . . 26
7. Best Practices — Application Management Current-State Assessment & Modernization Roadmap 27
8. Deliverables Summary — Application Management Current-State Assessment & Modernization Roadmap 28
Documentation . . . 28
PowerShell scripts (all READ-ONLY) . . . 29
Reusable templates and named artifacts . . . 29
What the buyer receives . . . 30
Outcome . . . 30
Appendix A — PowerShell Scripts 31

Executive Summary — Application Management Current-State Assessment & Modernization Roadmap

White-label · Intune Solutions

Almost every organization's application estate in Microsoft Intune grew rather than was designed. Apps arrived one at a time, were packaged by whoever was free, assigned to whichever group was handy, and never retired. The result is familiar and expensive: nobody can say how many apps are really deployed, install failures are tolerated as background noise, three versions of the same product run at once, and every conversation about "modernizing app management" stalls because there is no agreed picture of where things actually stand.

This is a complete, run-it-in-order package for **assessing the current state of application management and producing a prioritized modernization roadmap**. It is the front-door engagement — run it first, and the findings tell you which deeper workstreams to fund, in what order, and why. It scores nine dimensions of application management against a 1–5 maturity model with **observable evidence defined for every level**, so the scores are objective rather than opinion; it collects that evidence through structured interviews, an artifact request, and five **read-only** PowerShell scripts that read the tenant directly; and it turns the gaps into a findings register and a three-wave roadmap. It is written for an administrator with no prior assessment experience.

What you get

• A plain-language Solution Overview of the maturity model — nine dimensions (Governance & Ownership,

Packaging Standards, Deployment & Rings, Patching & Version Currency, Security & Application Control, Lifecycle & Retirement, Monitoring & Reporting, Support Model, Tooling & Migration Debt) × five levels (Initial → Optimized), each with an evidence table.

• A Requirements Guide with verify / implement-if-missing for every prerequisite, built around

least-privilege **read-only** access (Intune Read Only Operator + Graph `.Read.All` scopes).

• An Implementation Guide of eight ordered phases — charter, access, artifacts, interviews, technical

collection, evidence-based scoring, findings, and the roadmap & executive readout.

• Five **read-only** PowerShell scripts: app estate inventory & classification, assignment sprawl &

conflict analysis, install-failure & version-drift baseline, unmanaged & sideloaded app detection, and a maturity-evidence aggregator that emits a scored data pack.

• Reusable, named templates: the scoring sheet, findings register, risk matrix, roadmap one-pager, and

executive readout.

• Testing and Troubleshooting guides that keep the assessment honest, evidence-backed, and repeatable.

What this delivers

A named, reusable set of artifacts — the `APPASMT – Maturity Model (9 Dimensions x 5 Levels)`, the `APPASMT – Scoring Sheet` (a 1–5 score per dimension with the evidence behind it), the `APPASMT – Findings Register` (each gap rated for business impact and risk), and the `APPASMT – Roadmap One-Pager` (Quick Wins → Foundational → Strategic, plotted on effort vs impact with dependencies mapped and a 30/60/90 + 12-month view). The assessment measures and sequences; the detailed remediation build for any finding is a separate, downstream workstream.

The standard deployment model

Assess → **Design** → **Configure** → **Pilot** → **Validate** → **Expand.** This product owns the **Assess** stage and produces the **Design** input every downstream workstream consumes.

Outcome

A defensible answer to "how good is our application management, and what do we fix first?" — scored per dimension, rated for risk, and sequenced into a roadmap you can re-run each year to prove the trend and the return on the investment.

1. Solution Overview — Application Management Current-State Assessment & Modernization Roadmap

What this solution is

Almost every organization running Microsoft Intune has an application estate that grew rather than was designed. Apps arrived one at a time, packaged by whoever was available, assigned to whichever group was handy, and never retired. The result is familiar: nobody can say how many apps are actually deployed, install failures are tolerated as background noise, three versions of the same product are live at once, and every conversation about "modernizing app management" stalls because there is no agreed picture of where things actually stand.

This solution is a structured, repeatable assessment that produces that picture — and then turns it into a sequenced modernization roadmap. It is the front-door engagement: run it first, and the findings tell you which deeper workstreams to fund, in what order, and why.

It delivers four things:

1. A **capability maturity model** — nine dimensions of application management, scored 1–5, with **observable evidence** defined for every level so scoring is objective rather than opinion.
1. A **discovery method** — an interview guide (who to ask, what to ask), an artifact request list, and a **read-only technical data collection** from Intune and Microsoft Graph that evidences the scores.
1. A **gap analysis** — every gap becomes a written finding with business impact, a risk rating, and a remediation direction with a sized effort.
1. A **roadmap** — findings sequenced into three waves (Quick Wins → Foundational → Strategic) plotted

on effort vs impact, with dependencies mapped and a 30/60/90-day plus 12-month view, delivered in an executive readout.

What this solution is NOT

This is an **assessment and sequencing** product. It tells you what is broken, how badly, and in what order to fix it. It does **not** contain the implementation detail for the fixes themselves — those are separate, deeper engagements. Specifically, this package does not teach:

- How to build an application governance operating model (intake, roles, decision forum, change tiers).
- How to write packaging and deployment standards (detection rules, requirement rules, PSADT wrappers,

supersedence chains).

- How to run an application retirement and decommissioning program.
- How to remediate Win32 app uninstall and removal failures.
- How to build application estate monitoring, reporting, and supportability tooling.

Where a finding points at one of those, this solution states the **remediation direction and the effort size** — and stops there. That boundary is deliberate: an assessment that drifts into implementation detail stops being an assessment and becomes an unread 300-page document.

The maturity model — `APPASMT – Maturity Model (9 Dimensions x 5 Levels)`

This is the core named artifact. It is reused verbatim in the Requirements Guide, the Implementation Guide, the Testing Guide, the Troubleshooting Guide, and every script.

The nine dimensions

#	Dimension	What it measures
D1	Governance & Ownership	Whether every app has a named accountable owner, an intake path, and an approval record
D2	Packaging Standards	Whether apps are packaged to a written standard — naming, detection, requirements, install behaviour
D3	Deployment & Rings	Whether deployments follow defined rings with gates, or go straight to All Devices
D4	Patching & Version Currency	Whether app versions are kept current on a defined cadence, and how much drift exists

#	Dimension	What it measures
D5	Security & Application Control	Whether app execution is controlled (WDAC/AppLocker), signed, and security-reviewed before deployment
D6	Lifecycle & Retirement	Whether apps are reviewed, superseded, and actually removed when no longer needed
D7	Monitoring & Reporting	Whether install success, failure, and estate composition are measured and reported to someone
D8	Support Model	Whether app tickets have a defined route, runbooks, and escalation to a named owner or vendor
D9	Tooling & Migration Debt	Whether the estate is genuinely Intune-native, or still carries SCCM/co-management/manual remnants

The five levels

Level	Name	General meaning
L1	Initial	Ad hoc. Undocumented, person-dependent, no repeatable process.
L2	Repeatable	A process exists in practice but is unwritten and inconsistently applied.
L3	Defined	Written, published, and followed for new work. Legacy backlog not yet remediated.
L4	Managed	Measured. Metrics exist, are reviewed on a cadence, and drive corrective action.
L5	Optimized	Automated and continuously improved. Exceptions are rare, visible, and closed.

The generic level names above are deliberately not the scoring criteria. **Each dimension has its own evidence table** — the exact observable artifact or data signal required to claim each level — in Implementation Step 5. That is what makes the score defensible: an assessor cannot award L3 on D2 without producing the written packaging standard document.

How the assessment runs

The engagement is eight ordered phases (Implementation Guide, Steps 1–8):

1. **Charter** — agree scope, dimensions, target levels, stakeholders, and the readout date.
2. **Access** — obtain least-privilege **read-only** tenant access and verify it.
3. **Artifacts** — request and collect the documentary evidence (`APPASMT – Artifact Request List`).
4. **Interviews** — run the structured interviews (`APPASMT – Discovery Interview Guide`).
5. **Technical collection** — run the five read-only scripts to build the `APPASMT – Evidence Pack`.
6. **Score** — apply the evidence tables to produce the `APPASMT – Scoring Sheet`.
7. **Findings** — convert gaps into the `APPASMT – Findings Register` with impact and risk ratings.
8. **Roadmap & readout** — sequence findings into the `APPASMT – Roadmap One-Pager` and present the

`APPASMT – Executive Readout`.

Everything is read-only

This assessment must not change the tenant. All five scripts use read-only Microsoft Graph scopes, and the recommended Intune role is **Read Only Operator**. Nothing is created, modified, assigned, or deleted. This matters commercially as much as technically: an assessment that changes production is an assessment nobody will let you run again, and a customer who has to schedule a change window will delay the engagement by weeks.

Named artifacts produced

These names are used verbatim throughout the documentation and the scripts:

Artifact	What it is
`APPASMT – Assessment Charter`	Scope, dimensions in play, target levels, stakeholders, dates, out-of-scope statement
`APPASMT – Maturity Model (9 Dimensions x 5 Levels)`	The scoring rubric with per-dimension evidence tables
`APPASMT – Artifact Request List`	The documents and exports requested from the customer
`APPASMT – Discovery Interview Guide`	Structured questions per stakeholder role
`APPASMT – Evidence Pack`	The folder of script outputs (CSV/JSON) that evidences every score
`APPASMT – Scoring Sheet`	The 9-dimension score table: current level, target level, gap, evidence citation

Artifact	What it is
`APPASMT – Findings Register`	One row per finding: dimension, description, evidence, business impact, risk rating, remediation direction, effort
`APPASMT – Roadmap One-Pager`	The three waves plotted on effort vs impact, with dependencies
`APPASMT – Wave Plan`	Wave 1 Quick Wins (0–30 days) · Wave 2 Foundational (30–90 days) · Wave 3 Strategic (90 days–12 months)
`APPASMT – Executive Readout`	The presentation deck and its mandatory executive summary

Benefits

• **A number, not a debate.** Nine dimensions, each scored 1–5 against observable evidence. Two assessors running this model on the same tenant reach the same scores.

• **Evidence, not survey answers.** Interviews describe intent; the scripts show reality. Where they disagree, the data wins, and that disagreement is itself a finding.

• **A defensible investment case.** Every finding carries a business impact and a risk rating, so the budget conversation is about consequences rather than preferences.

• **A sequenced plan, not a wish list.** Quick wins fund credibility; foundational work unblocks the strategic work. The roadmap makes the dependencies explicit.

• **Repeatable.** Re-run the same model in six or twelve months and the movement in the scores is the proof the investment worked.

Limitations — be honest with the buyer

• **The assessment measures configuration and process, not user satisfaction.** A D3 score of L4 means

rings are defined and measured; it does not prove users are happy with the change cadence.

• **Scores are as good as the access granted.** A dimension the assessor cannot read scores as "Insufficient Evidence," not as L1. Guessing a low score to fill a gap is the single most common way an assessment loses credibility.

• **Findings that depend on non-Intune systems are bounded.** Procurement systems, ITSM tooling, and

software asset management platforms are described from interview and artifact evidence only; this solution does not integrate with them.

• **Remediation is out of scope.** The roadmap sizes the work; it does not do it. Where a finding needs deep implementation detail, that is a separate engagement.

• **Effort estimates are ranges, not quotes.** Effort is sized in T-shirt bands (S/M/L/XL) with a day range per band. Converting a band into a fixed price requires the customer's own rates and constraints.

The standard deployment model

Assess → Design → Configure → Pilot → Validate → Expand.

This solution owns the **Assess** stage in full, and produces the **Design** input (the roadmap) that every downstream workstream consumes. Configure, Pilot, Validate, and Expand belong to the workstreams the roadmap sequences.

2. Requirements Guide — Application Management Current-State Assessment & Modernization Roadmap

Verify **every** requirement before you begin scoring. Each entry follows the same shape: **Description · Why required · How to verify · How to implement if missing.** Category-wide prerequisites live in `global-requirements-intune-solutions.md` — read that first; this guide adds only the assessment-specific prerequisites and does not repeat the seven.

This is a **read-only** assessment. The access requirements below are deliberately least-privilege: you need to *read* the tenant, never to change it.

2.1 Least-privilege read-only tenant access (Intune role: Read Only Operator)

Description: An assessor account granted the built-in Intune RBAC role **Read Only Operator**, plus the Entra role **Global Reader**. This combination reads every Intune app, assignment, policy, and device record and every Entra group and directory object the assessment needs — without any write capability.

Why required: The scores must reflect what exists in the tenant, not what a low-privilege account happens to be able to see. A dimension the account cannot read will false-score. Equally, granting the assessor a write-capable role (Intune Administrator, Application Manager) breaks the read-only guarantee that lets the customer approve the engagement quickly.

How to verify:

1. In the Intune admin center (`https://intune.microsoft.com`), go to **Tenant administration → Roles →

All roles → Read Only Operator → Assignments **and confirm the assessor account (or an Entra group it belongs to) is listed with an assignment scoped to** All devices / All users** (or the agreed scope).

1. In the Entra admin center (`https://entra.microsoft.com`), go to **Roles and administrators → Global

Reader → Assignments** and confirm the account is present.

1. Run `APPASMT-Estate-Inventory.ps1 -WhatIfAccessCheck` (the header block documents this switch); it

confirms the sign-in succeeds and reports which object types are readable.

How to implement if missing:

- Intune → **Tenant administration** → **Roles** → **All roles** → **Read Only Operator** → **Assignments** → **Assign** →

add the assessor account or a dedicated Entra group named `APPASMT – Assessors`, choose the scope groups (or All), and save.

- Entra → **Roles and administrators** → **Global Reader** → **Add assignments** → add the same account.
- Do **not** substitute Intune Administrator or Global Administrator "to save time" — the read-only

posture is a deliverable, not a convenience.

2.2 Microsoft Graph PowerShell SDK with read-only scopes consented

Description: A workstation with the **Microsoft Graph PowerShell SDK** installed and the assessor able to consent to the following **read-only** delegated scopes: `DeviceManagementApps.Read.All`, `DeviceManagementConfiguration.Read.All`, `DeviceManagementManagedDevices.Read.All`, `DeviceManagementRBAC.Read.All`, `Group.Read.All`, `Directory.Read.All`.

Why required: All five assessment scripts connect through Graph. Every scope ends in `.Read.All` — there is no `.ReadWrite` scope anywhere in this solution, which is what enforces the read-only guarantee at the API level regardless of the account's role.

How to verify:

- Run `Get-InstalledModule Microsoft.Graph` — it returns an installed version.
- Run `Connect-MgGraph -Scopes "DeviceManagementApps.Read.All","DeviceManagementConfiguration.Read.All","DeviceManagementManagedDevices.Read.All","DeviceManagementRBAC.Read.All","Group.Read.All","Directory.Read.All"`

and confirm it completes and, on first run, that an administrator consents to exactly these read scopes.

- Run `Get-MgContext` and confirm the `Scopes` list contains only `.Read.All` entries.

How to implement if missing:

- `Install-Module Microsoft.Graph -Scope CurrentUser` (or `-Scope AllUsers` from an elevated prompt).
- If your tenant requires admin consent for delegated scopes, have a Global Administrator approve the

read scopes once (Entra → **Enterprise applications** → **Microsoft Graph Command Line Tools** → **Permissions** → **Grant admin consent**, or accept the consent prompt on first `Connect-MgGraph`).

- **Government tenants:** connect with `Connect-MgGraph -Environment USGov` (GCC High) — see §2.7.

2.3 An agreed Assessment Charter (scope, dimensions, target levels)

Description: A signed one-pager, `APPASMT – Assessment Charter`, recording: which of the nine dimensions are in scope, the **target level** the organization is aiming for on each (not every organization needs L5 everywhere), the platforms in scope (Windows / macOS / iOS / Android), which business units or

device groups are included, what is explicitly **out of scope**, the stakeholder list, and the readout date.

Why required: A maturity score is meaningless without a target to compare it to. A gap is the distance between the current level and the agreed target — with no agreed target, there is no gap and no roadmap. The charter also protects the engagement from scope creep.

How to verify: The charter exists, names a target level per in-scope dimension, and is signed by the sponsor and the endpoint/security owners.

How to implement if missing: Use the `APPASMT – Assessment Charter` template (Deliverables). Default target levels: **L3 (Defined)** for D1, D2, D3, D6, D8; **L4 (Managed)** for D4, D5, D7; **L3** for D9 — then adjust with the customer before scoring.

2.4 Named stakeholders available for interview

Description: Confirmed access to the people who can answer for each dimension: **application owners / business app sponsors**, **endpoint engineers** (the people who package and deploy), **security / endpoint security**, **service desk / support lead**, and **procurement / software asset management**.

Why required: The technical data shows *what* is configured; the interviews explain *why*, *who owns it*, and *what process produced it*. Dimensions D1 (Governance), D8 (Support), and D9 (Migration debt) cannot be scored from tenant data alone — they need the interviews.

How to verify: Each role has a named person and a booked interview slot in the assessment window.

How to implement if missing: Work with the sponsor to identify the person actually doing each job (not the org-chart owner). Where a role does not exist — e.g. "no one owns application security review" — that absence is itself a finding, recorded against the relevant dimension.

2.5 Documentary artifacts requested and a place to store the Evidence Pack

Description: The `APPASMT – Artifact Request List` sent to the customer (packaging standard docs, intake/approval records, ring definitions, patch cadence policy, WDAC/AppLocker policy, retirement records, support runbooks, any SCCM/co-management inventory), plus a secure location to hold the `APPASMT – Evidence Pack` (the script outputs) for the duration of the engagement.

Why required: Interviews claim processes exist; artifacts prove they do. A D2 (Packaging Standards) score of L3 requires the *actual written standard* to be produced — "we have one" is not evidence.

How to verify: The request list has been sent; a shared, access-controlled folder (customer or assessor tenant) exists for the Evidence Pack and collected artifacts.

How to implement if missing: Send the `APPASMT – Artifact Request List` at charter sign-off. Create the evidence folder before the technical collection in Step 5. Never store tenant exports containing device or user identifiers in an uncontrolled location.

2.6 A workstation that meets the collection prerequisites

Description: A Windows workstation (or PowerShell 7 on any supported OS) with outbound HTTPS to Microsoft Graph, PowerShell 5.1 or 7.x, and permission to run local scripts (`Set-ExecutionPolicy -Scope Process Bypass` for the session, or signed scripts).

Why required: The five scripts run locally against Graph. Corporate proxies or execution-policy lockdown are the two most common reasons collection stalls on day one.

How to verify: From the workstation, `Connect-MgGraph` succeeds and `Invoke-MgGraphRequest -Method GET -Uri 'https://graph.microsoft.com/v1.0/deviceManagement/mobileApps?$top=1'` returns data.

How to implement if missing: Allow the Graph endpoints through the proxy, run PowerShell as a normal user with `-ExecutionPolicy Bypass` for the process, and confirm TLS 1.2 is enabled.

2.7 Government (GCC High / DoD) — confirm the sovereign-cloud endpoints

Description: If the tenant is a US-sovereign-cloud tenant (GCC High or DoD), the portals and Graph endpoint differ from Commercial and must be targeted explicitly.

Why required: Running the Commercial endpoints against a Government tenant simply fails to authenticate; the assessor must point the tools at the `.us` sovereign endpoints.

How to verify: Confirm with the customer which cloud the tenant is in. For GCC High, the admin portals are `https://intune.microsoft.us` (Intune) and `https://portal.azure.us` (Azure/Entra), and Graph is `https://graph.microsoft.us`.

How to implement if missing: Connect Graph with `Connect-MgGraph -Environment USGov` (GCC High) or `-Environment USGovDoD` (DoD). Each script's `.NOTES` documents the switch. Where a specific Intune feature's Government parity is uncertain, **verify for GCC High** against Microsoft's US-sovereign-cloud documentation rather than assuming Commercial behaviour — the read-only assessment logic itself is identical across clouds; only the endpoints change.

Requirements checklist

- **2.1** Assessor has **Read Only Operator** (Intune) + **Global Reader** (Entra) — verified
- **2.2** Microsoft Graph PowerShell SDK installed; **read-only** scopes consented (`Get-MgContext` shows only `.Read.All`)
- **2.3** `APPASMT – Assessment Charter` signed with a target level per in-scope dimension
- **2.4** Named stakeholders (app owners, endpoint eng, security, service desk, procurement) booked
- **2.5** `APPASMT – Artifact Request List` sent; secure Evidence Pack folder created
- **2.6** Collection workstation reaches Graph; scripts can run
- **2.7** For Government tenants: sovereign-cloud endpoints confirmed and `-Environment` set

Global Requirements — Intune Solutions (category-level)

These requirements apply to **every** solution in the **Intune Solutions** category. They are published once at the category level so individual listings stay small. Confirm these before deploying *any* Intune solution. Each entry: **Description · Why · Verify · Implement-if-missing.**

G1. Microsoft Entra ID tenant

Description: An active Microsoft Entra ID (Azure AD) tenant. **Why:** It is the identity backbone for every cloud and Intune-managed resource. **Verify:** Sign in to `https://entra.microsoft.com` as an administrator. **Implement:** Create a Microsoft 365/Azure subscription, which provisions a tenant.

G2. Microsoft Intune subscription

Description: An Intune-bearing license plan. **Why:** Intune delivers all configuration, compliance, and security policies. **Verify:** `https://intune.microsoft.com` loads; **Tenant administration** → **Tenant status** shows active. **Implement:** Assign Microsoft 365 E3/E5, Business Premium, or standalone Intune Plan 1.

G3. Administrative roles

Description: Appropriate Entra/Intune admin roles for the operator. **Why:** Creating policies and enrollment settings requires elevated rights. **Verify:** Entra → **Roles and administrators**; confirm **Intune Administrator**. **Implement:** Assign **Intune Administrator**; reserve Global Administrator for tenant-wide changes.

G4. Device enrollment in Intune

Description: Targeted Windows devices are enrolled and manageable. **Why:** Policies only reach enrolled devices. **Verify:** Intune → **Devices** → **All devices**; the device appears and is **compliant**. **Implement:** Enable **Automatic Enrollment** and join devices to Entra ID.

G5. Pilot security group

Description: A Microsoft Entra security group for pilot/targeted users or devices. **Why:** Every solution is piloted before fleet rollout. **Verify:** Entra → **Groups**; the pilot group exists. **Implement:** Entra → **Groups** → **New group** → **Security** → add members.

G6. Supported Windows baseline

Description: Windows 10 21H2+ or Windows 11 with current updates. **Why:** Modern Intune features assume a patched, supported OS. **Verify:** `winver`, or Intune → device → **Hardware**. **Implement:** Deploy **Update rings** or upgrade to Windows 11.

G7. Network access to Microsoft cloud endpoints

Description: Outbound HTTPS to Microsoft 365, Entra, and Intune service URLs. **Why:** Enrollment, policy sync, and authentication depend on these endpoints. **Verify:** From a device, reach `https://login.microsoftonline.com` and `https://enrollment.manage.microsoft.com`. **Implement:** Allow Microsoft's published Office 365/Intune URL & IP ranges through proxy/firewall.

The standard deployment model (all solutions): Assess → Configure → Pilot → Validate → Expand.

3. Implementation Guide — Application Management Current-State Assessment & Modernization Roadmap

Follow these steps **in order**. This is an **assessment**, not a configuration change: every step is read-only or documentary. The tenant is never modified. The named artifacts below are reused verbatim in the Testing Guide, the Troubleshooting Guide, and the scripts.

> ***Model:*** *Charter → Access → Collect (artifacts, interviews, technical) → Score against evidence → Turn gaps into findings → Sequence into a roadmap → Readout. This owns the* ***Assess*** *stage of the house pattern and produces the* ***Design*** *input (the roadmap) for every downstream workstream.*

Object	Name
Charter	`APPASMT – Assessment Charter`
Maturity model (rubric)	`APPASMT – Maturity Model (9 Dimensions x 5 Levels)`
Artifact request	`APPASMT – Artifact Request List`
Interview guide	`APPASMT – Discovery Interview Guide`
Evidence folder	`APPASMT – Evidence Pack`
Scored output	`APPASMT – Scoring Sheet`
Findings	`APPASMT – Findings Register`
Roadmap	`APPASMT – Roadmap One-Pager` + `APPASMT – Wave Plan`
Readout	`APPASMT – Executive Readout`

Step 1 — Charter the assessment (scope & targets)

1. Complete `APPASMT – Assessment Charter` with the sponsor: which of the nine dimensions are in scope,

the **target level (L1–L5) per dimension**, the platforms in scope (Windows / macOS / iOS / Android), the business units or device groups included, the explicit out-of-scope statement, the stakeholder list, and the readout date.

1. Confirm the **default targets** (Requirement 2.3) or adjust them with the customer. Record who signed.
2. **Pass/fail:** the charter is signed and every in-scope dimension has a named target level.

Step 2 — Obtain and verify read-only access (Access)

1. Confirm the assessor account holds **Read Only Operator** (Intune) and **Global Reader** (Entra) per

Requirement 2.1.

1. Install/confirm the Microsoft Graph PowerShell SDK and consent the **read-only** scopes per

Requirement 2.2. **Government tenants:** add `-Environment USGov` (Requirement 2.7).

1. Run `APPASMT-Estate-Inventory.ps1 -WhatIfAccessCheck`. Confirm it signs in, reports the granted

scopes as read-only, and lists which object types are readable.

1. **Pass/fail:** sign-in succeeds; `Get-MgContext` shows only `.Read.All` scopes; all in-scope object

types are readable, or a documented reason (role/licensing) exists for any that are not.

Step 3 — Request and collect documentary artifacts (Collect)

1. Send `APPASMT – Artifact Request List` (if not already sent at charter). It requests, at minimum:

the written packaging standard, application intake and approval records, deployment ring definitions, the patch/version-currency cadence policy, the WDAC/AppLocker policy, application retirement records, support runbooks and the ticket-routing map, the software asset management / license position, and any SCCM or co-management inventory.

1. Save every artifact received into the `APPASMT – Evidence Pack` folder, named by dimension

(e.g. `D2 - Packaging-Standard.docx`).

1. **Pass/fail:** each requested artifact is either collected or explicitly recorded as "not provided /

does not exist" — the absence is a data point, not a blank.

Step 4 — Run the structured interviews (Collect)

1. Using `APPASMT – Discovery Interview Guide`, interview each stakeholder role. Ask, per role:

• **Application owners / sponsors:** How is a new app requested and approved? Who signs off? Who is

accountable when it breaks? (evidences D1)

• **Endpoint engineers:** Walk me through packaging one app end to end. Is there a written standard?

How do you deploy — rings or straight to All Devices? How do you keep versions current? How do you retire an app? (evidences D2, D3, D4, D6)

• **Security / endpoint security:** Is app execution controlled (WDAC/AppLocker)? Are apps security-

reviewed and signature-checked before deployment? (evidences D5)

• **Service desk / support lead:** When an app ticket arrives, where does it go? Is there a runbook?

Who is the escalation owner or vendor contact? (evidences D8)

• **Procurement / SAM:** How do you know what's licensed vs installed? Is there version/licence drift?

(evidences D4, D9)

1. Capture answers as notes in the Evidence Pack, tagged by dimension.

2. **Pass/fail:** every in-scope dimension has interview evidence from at least the role that owns it.

Step 5 — Collect the technical evidence (Collect)

Run the five **read-only** scripts. Each writes CSV/JSON into `APPASMT – Evidence Pack`. Run them in this order:

1. `APPASMT-Estate-Inventory.ps1` — the app estate: counts by app type (Win32, MSI/LOB, store, web

link, iOS/Android), publisher, and assignment intent. Builds the baseline "how many apps, of what kind" picture. (evidences D1, D9)

1. `APPASMT-Assignment-Analysis.ps1` — assignment sprawl and conflicts: apps assigned to overlapping

groups, apps assigned directly to All Devices/All Users, apps with both required and available intents colliding, and orphaned assignments (groups with no members). (evidences D1, D3)

1. `APPASMT-Install-Health.ps1` — install-failure and version-drift baseline: per-app install

success/failure/pending counts from device app-status, and version spread per app title (how many distinct versions of the same product are live). (evidences D4, D7)

1. `APPASMT-Unmanaged-Detection.ps1` — unmanaged and sideloaded apps: from `detectedApps`, the software

present on managed devices that is **not** deployed by Intune (manually installed / sideloaded), and apps with no assignment that are nonetheless detected. (evidences D5, D6, D9)

1. `APPASMT-Maturity-Evidence.ps1` — the aggregator: reads the four outputs above plus signals for

app-control (WDAC/AppLocker configuration profiles), co-management/SCCM workload signals, and app configuration/protection policies, and emits a single scored data pack (`APPASMT-EvidencePack.json`) with a **suggested** level and the signal behind it for each of the nine dimensions. (evidences all)

For a Government tenant, pass `-Environment USGov` to each script (documented in each `.NOTES`).

Pass/fail: all five scripts complete and the Evidence Pack contains a signal for each dimension, or an explicit "insufficient data" marker where a signal could not be read.

Step 6 — Score each dimension against the evidence tables (Score)

Open `APPASMT – Scoring Sheet` and, for each dimension, award the level whose **evidence table** is fully satisfied by the collected evidence (artifacts + interviews + `APPASMT-Maturity-Evidence.ps1` suggestion). A level requires **all** of its evidence to be present; if only some is, the dimension scores the level below. Where evidence is missing entirely, mark **"Insufficient Evidence"** — never guess a low score.

The nine evidence tables (`APPASMT – Maturity Model (9 Dimensions x 5 Levels)`):

D1 — Governance & Ownership

Level	Observable evidence required
L1	No app owner list; apps requested informally; no approval record anywhere

Level	Observable evidence required
L2	An owner is *usually* known but recorded in people's heads or scattered emails; no single intake
L3	A written intake path and an owner register exist and are used for new apps; approvals are recorded
L4	Ownership and approval are reviewed on a cadence; orphaned-app metrics exist and are actioned
L5	Intake/approval is automated (workflow/ITSM), ownership is enforced, orphans are auto-flagged and closed

D2 — Packaging Standards

Level	Observable evidence required
L1	No written standard; each engineer packages their own way; naming is inconsistent (script confirms name entropy)
L2	An unwritten "house way" is followed by most engineers; detection rules exist but vary
L3	A **written packaging standard document exists** (produced as artifact) covering naming, detection, requirement rules, install behaviour, and is followed for new work
L4	Packaging conformance is measured (e.g. QA checklist, sample audits) and non-conformance is corrected
L5	Packaging is templated/automated (e.g. PSADT + pipeline), conformance is enforced pre-publish

D3 — Deployment & Rings

Level	Observable evidence required
L1	Apps deployed straight to All Devices/All Users; no rings (script confirms All-Devices assignments)
L2	An informal "test on a few machines first" habit; no defined ring groups
L3	**Defined ring groups** (e.g. pilot → early → broad) exist and new deployments follow them; gates are documented
L4	Ring progression is gated on measured install success and reviewed before each promotion

Level	Observable evidence required
L5	Ring promotion is automated on success thresholds; rollback is automated

D4 — Patching & Version Currency

Level	Observable evidence required
L1	No cadence; apps updated only when broken; high version spread (script: multiple live versions per title)
L2	Ad hoc updates when someone notices; some products current, many stale
L3	A **written cadence** exists and priority apps are kept current; supersedence is used
L4	Version currency is measured and reported; drift is tracked and closed to target
L5	Updating is automated (e.g. WinGet/patch tooling + supersedence); drift is near-zero and self-correcting

D5 — Security & Application Control

Level	Observable evidence required
L1	No app-execution control (no WDAC/AppLocker profile — script confirms); no pre-deployment security review
L2	App control discussed or piloted on a few devices; security review is informal
L3	**WDAC or AppLocker is deployed** to a meaningful scope and apps are security-reviewed before deployment
L4	App control is enforced fleet-wide in enforce mode, measured, and exceptions are tracked
L5	App control is fully enforced, signed-catalog managed, integrated with security monitoring

D6 — Lifecycle & Retirement

Level	Observable evidence required
L1	Apps are never removed; retired products still assigned; no review (script: assigned apps with no installs)
L2	Occasional cleanups when storage or licence pressure forces it; no process

Level	Observable evidence required
L3	A **written review/retirement process** exists and superseded apps are removed on a cadence
L4	Retirement is measured (apps retired vs candidates) and reported
L5	Lifecycle is automated end-to-end (auto-flag unused, supersede, uninstall, decommission)

D7 — Monitoring & Reporting

Level	Observable evidence required
L1	Nobody looks at install success/failure; no reporting (interview confirms)
L2	Someone checks the Intune app dashboard reactively when a rollout is questioned
L3	**A regular report** of install success/failure and estate composition is produced and read by an owner
L4	Metrics feed a reviewed scorecard with thresholds; failures trigger action
L5	Monitoring is automated with alerting and trend dashboards; SLAs on install success exist

D8 — Support Model

Level	Observable evidence required
L1	App tickets go wherever; no runbooks; one engineer "just knows" (interview confirms bus-factor risk)
L2	Tickets usually reach the right person but by tribal knowledge, not a defined route
L3	**A defined ticket-routing map and runbooks exist**, with a named escalation owner or vendor per app tier
L4	Support performance is measured (time-to-resolve by app), reviewed, and runbooks are maintained
L5	Support is proactive (known-issue automation, self-heal), integrated with monitoring

D9 — Tooling & Migration Debt

Level	Observable evidence required
L1	Heavy SCCM/co-management or manual-install dependence; Intune is partial (script: co-mgmt/detectedApps signals)
L2	Migration to Intune-native is underway but stalled; significant remnants remain
L3	The estate is **majority Intune-native**; remaining SCCM/manual items are inventoried with a plan
L4	Migration is measured to completion; residual debt is tracked and closing
L5	Fully Intune-native; no SCCM/manual app-delivery remnants; tooling is consolidated

For each dimension record in the `APPASMT – Scoring Sheet`: **current level**, **target level** (from the charter), **gap** (target – current), and the **evidence citation** (which artifact / interview / script output justifies the score).

Pass/fail: every in-scope dimension has a current level (or "Insufficient Evidence"), a target, a gap, and a cited evidence source.

Step 7 — Turn gaps into findings (Findings)

1. For every dimension with a gap > 0, write one or more rows in the `APPASMT – Findings Register`. Each

finding has: dimension, a plain-language description of what is wrong, the **evidence** (cited), the **business impact** (what this costs or risks — e.g. "3 live versions of the PDF reader means unpatched RCE exposure and support confusion"), a **risk rating** (Critical / High / Medium / Low using the `APPASMT – Risk Matrix`: likelihood × impact), the **remediation direction** (what kind of work closes it — described, not built), and a **sized effort** (S/M/L/XL band with a day range).

1. **Do not re-teach the remediation.** Where a finding points at deeper work — governance operating

model, packaging standards, retirement program, uninstall remediation, or monitoring tooling — state the direction and the effort and reference that the detailed build is a separate workstream. Example: "Direction: establish a written packaging standard and QA gate (foundational packaging-standards workstream). Effort: L (10–15 days)." Nothing more.

1. **Pass/fail:** every gap is represented by at least one finding with all fields populated; no finding

contains build-level step-by-step detail.

Step 8 — Sequence the roadmap and run the readout (Roadmap → Readout)

1. Plot every finding on `APPASMT – Roadmap One-Pager` — an **effort (x) vs impact (y)** chart — and

assign each to a wave in `APPASMT – Wave Plan`:

• **Wave 1 — Quick Wins (0–30 days):** high impact, low effort. Things that build credibility and

reduce risk fast (e.g. remove orphaned assignments, retire clearly-dead apps, turn on the app install report). These fund the trust for the bigger waves.

• **Wave 2 — Foundational (30–90 days):** the enablers other work depends on (e.g. write the packaging

standard, define deployment rings, establish the ownership register). Usually medium-to-high effort, high leverage.

• **Wave 3 — Strategic (90 days–12 months):** the high-effort transformation (e.g. WDAC enforcement,

full SCCM→Intune migration, automated lifecycle). Depends on the foundational wave being in place.

1. Draw the **dependencies** between waves explicitly — a strategic item that depends on a foundational

one cannot start early. Map them on the one-pager as arrows.

1. Produce the **30/60/90-day plus 12-month view**: what completes by day 30, day 60, day 90, and by

month 12, tied to the waves.

1. Build `APPASMT – Executive Readout`. Its executive summary **must** contain, in one page: the overall

maturity band (average of the nine scores), the three findings with the highest risk rating, the single most important dependency, the Wave 1 quick wins the customer can start next week, and the headline effort for the full roadmap. Present current state → target state → sequenced plan; get sign-off on the roadmap.

1. **Pass/fail:** every finding sits in exactly one wave; dependencies are drawn; the 30/60/90 + 12-month

view exists; the executive summary contains all five mandatory elements.

Order of operations

`Charter (targets) → verify read-only access → collect artifacts → interviews → run 5 read-only scripts → score against the 9 evidence tables → gaps become findings (impact + risk + effort) → sequence into 3 waves on effort-vs-impact with dependencies → executive readout`

5. Testing Guide — Application Management Current-State Assessment & Modernization Roadmap

The purpose of these tests is different from a configuration solution's. There is nothing deployed to verify. Instead, these tests prove the **assessment itself is trustworthy**: that every score is evidence-backed, that the method is repeatable (a second assessor reaches the same scores), and that the outputs (findings, roadmap, readout) are complete and internally consistent. Run each test at the stage named; the pass/fail gates at the end must all pass before the readout.

Test 1 — Access is genuinely read-only

Do: After connecting, run `Get-MgContext` and `APPASMT-Estate-Inventory.ps1 -WhatIfAccessCheck`. **Expected:** `Get-MgContext` shows scopes ending only in `.Read.All` — no `.ReadWrite` scope anywhere. The access check confirms the app estate is readable. No script in the

pack contains a `New-`, `Set-`, `Update-`, `Remove-`, or `Invoke-*` write call. **Why it matters:** the read-only guarantee is a deliverable. If any write scope is present, stop and reduce the grant before proceeding — the customer approved a read-only assessment.

Test 2 — Every dimension has evidence, or an explicit gap

Do: After Step 5, open the `APPASMT – Evidence Pack` and `APPASMT-EvidencePack.json`. **Expected:** Each of the nine dimensions has either a signal (script output, artifact, or interview note) or an explicit **"Insufficient Evidence"** marker. No dimension is silently blank. **Why it matters:** a blank dimension read as L1 is a fabricated finding. "Insufficient Evidence" is an honest, actionable result (it tells the customer to grant access or produce the artifact).

Test 3 — Every score cites its evidence

Do: Review the `APPASMT – Scoring Sheet`. For each dimension, follow the evidence citation to the artifact, interview note, or script output it names. **Expected:** Every current-level score points to a real, locatable piece of evidence. Spot-check three dimensions and confirm the evidence actually supports the level awarded per the evidence table in Implementation Step 5. **Why it matters:** a score you cannot trace to evidence is an opinion. The whole value of the model is that the score is defensible.

Test 4 — The score matches the evidence table (repeatability spot-check)

Do: Pick two dimensions — ideally one high and one low. Independently re-award the level from the evidence, using only the evidence table (Implementation Step 5), without looking at the recorded score. **Expected:** The independently re-derived level matches the recorded level. If it does not, the evidence table was applied inconsistently — reconcile and correct. **Why it matters:** repeatability is the test of objectivity. If two passes over the same evidence disagree, the model is being scored on opinion, not evidence.

Test 5 — Interview claims and technical data are reconciled

Do: Compare the interview evidence to the script evidence for D3 (rings) and D4 (currency). **Expected:** Where they agree, the score is confident. Where they disagree (e.g. "we always use rings" but `APPASMT-Assignment-Analysis.ps1` shows All-Devices assignments), the disagreement is **recorded as a finding**, not silently resolved in favour of one side. **Why it matters:** the gap between what people believe and what the tenant actually does is often the single most valuable finding in the whole assessment.

Test 6 — Every gap becomes a finding

Do: Cross-check the `APPASMT – Scoring Sheet` against the `APPASMT – Findings Register`. **Expected:** Every dimension with a gap > 0 has at least one finding. Each finding has all fields populated: dimension, description, cited evidence, business impact, risk rating, remediation direction, and effort band. No finding contains build-level step-by-step instructions. **Why it matters:** a gap with no finding is a gap the roadmap will miss; a finding that re-teaches implementation breaks the scope boundary that keeps this an assessment.

Test 7 — Every finding lands in exactly one wave, with dependencies drawn

Do: Review the `APPASMT – Roadmap One-Pager` and `APPASMT – Wave Plan`. **Expected:** Every finding sits in exactly one of Wave 1 (Quick Wins), Wave 2 (Foundational), or Wave 3 (Strategic). Each strategic item that depends on a foundational item has the dependency drawn. No Wave 3 item is scheduled to start before the Wave 2 item it depends on. **Why it matters:** a roadmap with unsequenced dependencies is a wish list. The dependency arrows are what make it a plan.

Test 8 — The executive summary contains all five mandatory elements

Do: Read the executive summary page of `APPASMT – Executive Readout`. **Expected:** It contains, on one page: (1) the overall maturity band, (2) the three highest-risk findings, (3) the single most important dependency, (4) the Wave 1 quick wins startable next week, and (5) the headline effort for the full roadmap. **Why it matters:** the executive summary is the only page most sponsors read. If it is missing any of the five, the decision-maker cannot make the decision the assessment exists to enable.

Test 9 — The assessment is repeatable end to end

Do: Confirm the Evidence Pack, Scoring Sheet, Findings Register, and Roadmap are all retained and named per this solution, and that re-running the five scripts against the same tenant reproduces the same summary counts (within the noise of ongoing tenant activity). **Expected:** The artifacts are complete and named; a re-run reproduces the machine signals. The engagement can be re-run in 6–12 months to measure the trend. **Why it matters:** the trend across re-runs is the proof the modernization investment worked — but only if the first run was captured in a form the second run can be compared to.

Pass/fail gate

Gate	Requirement
Read-only	Test 1 passes — only `.Read.All` scopes; no write calls anywhere
Evidence complete	Tests 2–3 pass — every dimension has evidence or an explicit gap; every score cites its source
Objective & reconciled	Tests 4–5 pass — independent re-scoring matches; interview vs data disagreements are findings
Findings complete	Test 6 passes — every gap is a fully-populated finding; no implementation detail
Roadmap sound	Tests 7–8 pass — findings waved with dependencies; executive summary has all five elements
Repeatable	Test 9 passes — artifacts retained and named; scripts reproduce; trend re-run is possible

6. Troubleshooting Guide — Application Management Current-State Assessment & Modernization Roadmap

Assessments fail differently from deployments. Nothing breaks in production — instead the *evidence* comes up short, a stakeholder won't engage, or the tenant access is narrower than promised. Each entry below is **Symptom** → **Cause** → **Resolution**, and the guiding rule throughout is the same: when evidence is missing, record it honestly as a gap. Never invent a score to fill a hole.

Access and connection

`Connect-MgGraph` *fails or the scripts return no data*

Symptom: A script throws "Not connected," or returns zero apps despite the tenant clearly having apps. **Cause:** No Graph session, missing read scopes, or (common) the account has the **Intune Administrator** role but the delegated Graph scopes were never consented; or the tenant is Government and the Commercial endpoint was used. **Resolution:** Run `Connect-MgGraph` with the six read scopes from Requirement 2.2 and confirm with `Get-MgContext` that they are present and end in `.Read.All`. For a Government tenant, add `-Environment USGov` (GCC High) or `-Environment USGovDoD` (DoD). If your tenant requires admin consent for delegated permissions, have a Global Administrator consent once. Re-run `APPASMT-Estate-Inventory.ps1 -WhatIfAccessCheck`.

A whole dimension reads as empty / "insufficient evidence"

Symptom: `APPASMT-Maturity-Evidence.ps1` marks D5 (app control) or D9 (migration debt) as "Insufficient Evidence — could not read." **Cause:** The assessor account cannot read that object type — usually a scoped RBAC assignment or a missing scope (e.g. `DeviceManagementConfiguration.Read.All` for app-control policies), or the feature is genuinely absent/unlicensed. **Resolution:** Confirm the scope is consented and the **Read Only Operator** assignment is scoped to All (not a subset). If access is correct and the object still isn't there, the capability really is absent — score the dimension L1 with the evidence "no policy present," **not** "Insufficient Evidence." The two are different findings: one says "we can't see it," the other says "it isn't there." Only use "Insufficient Evidence" when you genuinely could not read.

The customer will only grant a scoped or time-limited role

Symptom: The assessor is given Read Only Operator scoped to one business unit, or access for 48 hours only. **Cause:** Reasonable customer caution on a first engagement. **Resolution:** Accept it and **record the scope limit in the charter and on every affected score.** A scorecard built on one business unit is valid for that unit — say so explicitly; do not present it as tenant-wide. For time-limited access, run all five scripts first (they are fast and read-only) to capture the Evidence Pack while access lasts; interviews and artifact review need no tenant access and can follow.

Data quality and scoring

The install-status or version-drift numbers look implausibly low

Symptom: `APPASMT-Install-Health.ps1` shows almost no failures, or no version drift, on an estate you expect to be messy. **Cause:** The `installSummary` beta endpoint can lag or return partial data, and the version-drift normalization strips version tokens from display names — apps whose names don't embed a version won't be grouped. **Resolution:** Treat the script output as a **signal, not a verdict.** Corroborate with the Intune portal (**Apps** → **Monitor** → **App install status**) for a sample, and with the version column in `APPASMT-Estate-Inventory.csv`. Where naming is inconsistent enough that drift can't be detected automatically, that inconsistent naming is itself a D2 (Packaging Standards) finding.

Interview answers contradict the technical data

Symptom: Engineering says "we always deploy in rings," but `APPASMT-Assignment-Analysis.ps1` shows apps assigned to All Devices. **Cause:** The intended process and the actual practice have diverged — the single most common (and valuable) assessment finding. **Resolution:** Do **not** quietly pick a side. Record the contradiction as a finding against D3, cite both the interview and the script output, and let the readout surface it. The gap between belief and reality is exactly what the assessment exists to reveal.

Two assessors score the same dimension differently

Symptom: A review of the `APPASMT – Scoring Sheet` finds inconsistent levels for the same evidence. **Cause:** The evidence table in Implementation Step 5 was applied loosely — someone awarded a level on partial evidence. **Resolution:** Re-apply the rule: a level requires **all** of its evidence; partial evidence scores the level below. Reconcile against the table, not against each other's opinion. If the table itself is ambiguous for this tenant, tighten the wording in the charter's model annex and re-score consistently.

Stakeholders and scope

A key stakeholder won't engage or keeps cancelling

Symptom: The service-desk lead or an app owner never makes the interview, so D8 (Support) or D1 (Governance) has no interview evidence. **Cause:** Competing priorities, or the role is genuinely unstaffed. **Resolution:** Escalate through the sponsor named in the charter — engagement is the sponsor's to enforce. If the role is unstaffed ("no one actually owns app security review"), that absence **is the finding** — record it against the dimension with "role unstaffed" as the evidence and a governance finding in the register. Do not leave the dimension blank; an unstaffed function is a low maturity score, honestly evidenced.

The customer pushes to expand scope mid-assessment

Symptom: "While you're in there, can you also assess configuration profiles / compliance / drivers?" **Cause:** The assessment is surfacing value and the customer wants more. **Resolution:** Hold the charter. Note the request as a candidate for a **follow-on** engagement and keep this assessment to the nine application-management dimensions. Scope creep is the most common way an assessment misses its readout date. Capture the extra appetite; don't absorb it silently.

A finding clearly needs deep implementation detail the customer asks you to include

Symptom: "Can you just put the packaging-standard steps / the retirement runbook / the WDAC config in the report?" **Cause:** The customer wants the fix, not just the diagnosis. **Resolution:** Keep the boundary. The report states the remediation **direction and effort**; the detailed build is a separate workstream (governance operating model, packaging standards, retirement program, uninstall remediation, or monitoring/reporting — each its own engagement). Record the demand as a strong signal to sequence that workstream early in the roadmap, and offer it as the natural next step.

Reporting integrity

The roadmap has findings with no wave, or waves with impossible ordering

Symptom: A finding appears in no wave, or a Wave 3 strategic item is scheduled before the Wave 2 foundational item it depends on. **Cause:** The sequencing step (Implementation Step 8) skipped a dependency or a finding. **Resolution:** Re-run the Testing Guide Tests 6–7: every gap → a finding, every

finding → exactly one wave, every dependency drawn. Fix the ordering so no item starts before its prerequisite. A roadmap that violates its own dependencies is not executable and will be rejected in the readout.

The executive summary is missing an element

Symptom: The sponsor asks a question the summary should have answered (usually "what's the total effort?" or "what do we start Monday?"). **Cause:** The one-page summary omitted one of the five mandatory elements. **Resolution:** Confirm all five are present (Testing Guide Test 8): overall band, the three highest-risk findings, the key dependency, the Wave 1 quick wins, and the headline effort. The executive summary is the only page many sponsors read — it must stand alone.

7. Best Practices — Application Management Current-State Assessment & Modernization Roadmap

- **Score from evidence, never from opinion.** Every level in the `APPASMT – Scoring Sheet` must cite

the artifact, interview note, or script output that justifies it. A maturity number nobody can trace to evidence is a survey dressed as an assessment — and the first hostile question in the readout will expose it.

- **"Insufficient Evidence" is a valid, honest result — use it.** A dimension you could not read or

whose artifact was never produced scores "Insufficient Evidence," not a guessed L1. Guessing a low score to avoid a blank is the fastest way to lose the room; the real finding is "we could not evidence this — grant access / produce the standard."

- **Keep it read-only, and say so up front.** The five scripts use only `.Read.All` scopes and the

Intune **Read Only Operator** role. Lead the engagement with that promise: it removes the change-window objection, shortens time-to-start by weeks, and means the customer will let you run it again.

- **Reconcile interviews against the tenant — the gap is the gold.** People describe the process they

intend; the tenant shows the process they run. When "we always pilot in rings" meets an All-Devices assignment, that contradiction is often the single most valuable finding in the assessment. Record it, don't smooth it over.

- **Set target levels before you score.** Not every organization needs L5 everywhere. Agree each

dimension's target in the `APPASMT – Assessment Charter` first — a score without a target has no gap, and a gap is what the roadmap is made of.

- **Size effort in bands, not false-precision quotes.** Use S/M/L/XL with a day range. An assessment

that hands over day-exact estimates it cannot stand behind loses more credibility than one that honestly says "L: 10–15 days, refine against your rates and constraints."

- **Rate risk on consequence, not on how annoying it is.** Use the `APPASMT – Risk Matrix`

(likelihood × business impact). Three live versions of a browser is Critical because it is unpatched RCE exposure, not merely untidy. Frame every finding in the language of consequence the sponsor cares about.

- **Sequence for momentum: quick wins fund the foundations.** Wave 1 exists to build credibility and

cut risk fast (remove orphaned assignments, retire dead apps, turn on the install report). Delivering visible wins in the first 30 days is what earns the mandate for the harder Wave 2 and Wave 3 work.

- **Respect the boundary — assess and sequence, do not re-teach the build.** Where a finding points at

governance, packaging standards, retirement, uninstall remediation, or monitoring tooling, state the direction and the effort and stop. The detailed build is a separate workstream. An assessment that drifts into implementation becomes a document nobody finishes reading.

- **Draw the dependencies explicitly.** A strategic item that depends on a foundational one cannot start

early — say so with an arrow on the `APPASMT – Roadmap One-Pager`. A roadmap without dependencies is a wish list; the arrows are what make it executable.

- **Write the executive summary for the one person who reads only it.** Overall band, the three

highest-risk findings, the key dependency, next-week quick wins, and the headline effort — on one page. If the sponsor can make the funding decision from that page alone, the assessment did its job.

- **Retain everything, named, for the re-run.** Keep the Evidence Pack, Scoring Sheet, Findings

Register, and Roadmap named exactly as this solution names them. The trend across a second run in 6–12 months is the proof the investment worked — but only if the first run was captured in a comparable form.

8. Deliverables Summary — Application Management Current-State Assessment & Modernization Roadmap

Documentation

Document	Purpose
Executive Summary	The business case and what is in the box
1. Solution Overview	What the assessment is — the 9-dimension × 5-level maturity model, the discovery method, the gap analysis, and the roadmap — with benefits and limitations
2. Requirements Guide	Every assessment-specific prerequisite with Verify / Implement-if-missing (least-privilege read-only access, read scopes, charter, stakeholders, Government endpoints)
Global Requirements (Intune Solutions)	Category-wide prerequisites, published once
3. Implementation Guide	The assessment run as eight ordered phases — charter, access, artifacts, interviews, technical collection, scoring against the nine evidence tables, findings, roadmap & readout

Document	Purpose
5. Testing Guide	How to prove the assessment is trustworthy — read-only, evidence-backed, repeatable, complete — with pass/fail gates
6. Troubleshooting Guide	Field issues — incomplete data, uncooperative stakeholders, tenant access gaps — and how to keep the assessment honest through them
7. Best Practices	Field-proven recommendations for a defensible, credible assessment

PowerShell scripts (all READ-ONLY)

Script	What it does
`APPASMT-Estate-Inventory.ps1`	Inventories and classifies the Intune app estate by type, publisher, and assignment intent; builds the baseline "how many apps, of what kind" picture (D1, D9). Also runs the read-only access check.
`APPASMT-Assignment-Analysis.ps1`	Finds assignment sprawl and conflicts — All-Devices/Users targeting, conflicting intents, orphaned assignments to empty groups (D1, D3).
`APPASMT-Install-Health.ps1`	Baselines per-app install success/failure/pending and the version-drift picture (how many live versions per title) (D4, D7).
`APPASMT-Unmanaged-Detection.ps1`	Detects unmanaged / sideloaded software — present on managed devices but not deployed by Intune — the shadow estate (D5, D6, D9).
`APPASMT-Maturity-Evidence.ps1`	Aggregates the four evidence files plus app-control / co-management / app-policy signals into a scored data pack (`APPASMT-EvidencePack.json`) with a suggested level and the signal behind each of the nine dimensions.

Every script uses Microsoft Graph read-only (`.Read.All`) scopes only, aligns to the Intune **Read Only Operator** role, and targets Government tenants with `-Environment USGov` (GCC High) / `USGovDoD` (DoD). No script writes to the tenant.

Reusable templates and named artifacts

The assessment produces a consistent, named artifact set (defined in the Solution Overview and used verbatim throughout): `APPASMT – Assessment Charter`, `APPASMT – Maturity Model (9 Dimensions x 5 Levels)`, `APPASMT – Artifact Request List`, `APPASMT – Discovery Interview Guide`, `APPASMT – Evidence Pack`, `APPASMT – Scoring Sheet`, `APPASMT – Findings Register`, `APPASMT – Risk Matrix`, `APPASMT – Roadmap One-Pager`, `APPASMT –`

`Wave Plan`, and `APPASMT – Executive Readout`.

What the buyer receives

- The full assessment documentation set as a Word document and PDF.
- All five read-only PowerShell scripts in a single ZIP.
- A repeatable maturity model, scoring method, findings register, and modernization roadmap following

the house model — **Assess** → **Design** → **Configure** → **Pilot** → **Validate** → **Expand** — where this product owns the **Assess** stage and produces the **Design** input for every downstream workstream, with explicit pass/fail gates and a read-only guarantee.

Outcome

A defensible, evidence-based answer to "how good is our application management, and what do we fix first?" — a 1–5 score across nine dimensions, a findings register rating business impact and risk, and a three-wave modernization roadmap plotted on effort vs impact with dependencies mapped and a 30/60/90 + 12-month view. Re-run the same model to prove the trend and the return on the investment.

Appendix A — PowerShell Scripts

Included in the accompanying ZIP. Each script's header documents full usage.

APPASMT-Assignment-Analysis.ps1

Purpose: Evidences dimensions D1 (Governance) and D3 (Deployment & Rings) by finding assignment sprawl and conflicts across the Intune app estate: apps assigned to All Devices / All Licensed Users (no ring discipline), apps carrying conflicting intents (required AND available/uninstall to overlapping audiences), apps assigned to groups that have zero members (orphaned assignments), and the overall assignment-per-app spread. READ-ONLY — Graph .Read.All scopes only. HOW TO RUN Connect-MgGraph -Scopes "DeviceManagementApps.Read.All","Group.Read.All" GCC High: add -Environment USGov powershell -ExecutionPolicy Bypass -File .\APPASMT-Assignment-Analysis.ps1 -OutDir .\APPASMT-EvidencePack EXPECTED RESULTS Console summary plus: APPASMT-Assignment-Findings.csv (one row per flagged app/assignment issue) APPASMT-Assignment-Summary.json (aggregate counts for the maturity aggregator) .NOTES Intune Solutions — application management assessment. READ-ONLY (analysis; no tenant writes). Least-privilege: Intune "Read Only Operator" + Graph *.Read.All. Government: -Environment USGov (GCC High) / USGovDoD (DoD); verify GCC High feature parity where uncertain. > [CmdletBinding()] param([string]$OutDir = ".\APPASMT-EvidencePack")

How to run: Connect-MgGraph -Scopes "DeviceManagementApps.Read.All","Group.Read.All" GCC High: add -Environment USGov powershell -ExecutionPolicy Bypass -File .\APPASMT-Assignment-Analysis.ps1 -OutDir .\APPASMT-EvidencePack EXPECTED RESULTS Console summary plus: APPASMT-Assignment-Findings.csv (one row per flagged app/assignment issue) APPASMT-Assignment-Summary.json (aggregate counts for the maturity aggregator) .NOTES Intune Solutions — application management assessment. READ-ONLY (analysis; no tenant writes). Least-privilege: Intune "Read Only Operator" + Graph *.Read.All. Government: -Environment USGov (GCC High) / USGovDoD (DoD); verify GCC High feature parity where uncertain. > [CmdletBinding()] param([string]$OutDir = ".\APPASMT-EvidencePack")

APPASMT-Estate-Inventory.ps1

Purpose: Builds the baseline picture of the application estate for the "APPASMT - Maturity Model (9 Dimensions x 5 Levels)" assessment: every Intune app classified by type (Win32, MSI/LOB, store, web link, iOS/Android, macOS), by publisher, and by assignment intent (required / available / uninstall / none). Feeds dimensions D1 (Governance) and D9 (Migration Debt). READ-ONLY: it reads Microsoft Graph with .Read.All scopes only and never changes the tenant. HOW TO RUN 1) Commercial tenant: Connect-MgGraph -Scopes "DeviceManagementApps.Read.All","Group.Read.All" 2) Government (GCC High) tenant: Connect-MgGraph -Environment USGov -Scopes "DeviceManagementApps.Read.All","Group.Read.All" 3) run: powershell -ExecutionPolicy Bypass -File .\APPASMT-Estate-Inventory.ps1 -OutDir .\APPASMT-EvidencePack Access-only check (no export): .\APPASMT-Estate-Inventory.ps1 -WhatIfAccessCheck EXPECTED RESULTS Console summary (apps by type, top publishers, assignment intent mix) and, unless -WhatIfAccessCheck, two files in the Evidence Pack: APPASMT-Estate-Inventory.csv (one row per app) APPASMT-Estate-Summary.json (aggregate counts consumed by APPASMT-Maturity-Evidence.ps1) .NOTES Intune Solutions — application management assessment. READ-ONLY (inventory/analysis; no tenant writes). Least-privilege: Intune "Read Only Operator" + Graph *.Read.All. Government: use -Environment USGov (GCC High) /

USGovDoD (DoD) on Connect-MgGraph; verify feature parity for GCC High where uncertain. > [CmdletBinding()] param([string]$OutDir = ".\APPASMT-EvidencePack", [switch]$WhatIfAccessCheck)

How to run: 1) Commercial tenant: Connect-MgGraph -Scopes "DeviceManagementApps.Read.All","Group.Read.All" 2) Government (GCC High) tenant: Connect-MgGraph -Environment USGov -Scopes "DeviceManagementApps.Read.All","Group.Read.All" 3) run: powershell -ExecutionPolicy Bypass -File .\APPASMT-Estate-Inventory.ps1 -OutDir .\APPASMT-EvidencePack Access-only check (no export): .\APPASMT-Estate-Inventory.ps1 -WhatIfAccessCheck EXPECTED RESULTS Console summary (apps by type, top publishers, assignment intent mix) and, unless -WhatIfAccessCheck, two files in the Evidence Pack: APPASMT-Estate-Inventory.csv (one row per app) APPASMT-Estate-Summary.json (aggregate counts consumed by APPASMT-Maturity-Evidence.ps1) .NOTES Intune Solutions — application management assessment. READ-ONLY (inventory/analysis; no tenant writes). Least-privilege: Intune "Read Only Operator" + Graph *.Read.All. Government: use -Environment USGov (GCC High) / USGovDoD (DoD) on Connect-MgGraph; verify feature parity for GCC High where uncertain. > [CmdletBinding()] param([string]$OutDir = ".\APPASMT-EvidencePack", [switch]$WhatIfAccessCheck)

APPASMT-Install-Health.ps1

Purpose: Evidences dimensions D4 (Patching & Version Currency) and D7 (Monitoring & Reporting) by baselining, per app, the install success / failure / pending / not-applicable counts from Intune's app install status, and the VERSION SPREAD per app title (how many distinct versions of the same product are live at once — the classic modernization red flag). READ-ONLY: it reads Graph reporting/status endpoints with .Read.All scopes only. HOW TO RUN Connect-MgGraph -Scopes "DeviceManagementApps.Read.All" GCC High: add -Environment USGov powershell -ExecutionPolicy Bypass -File .\APPASMT-Install-Health.ps1 -OutDir .\APPASMT-EvidencePack EXPECTED RESULTS Console summary plus: APPASMT-Install-Health.csv (per-app install success/failure/pending + failure rate) APPASMT-Version-Drift.csv (app titles with >1 live version) APPASMT-InstallHealth-Summary.json .NOTES Intune Solutions — application management assessment. READ-ONLY (reporting/analysis; no tenant writes). Least-privilege: Intune "Read Only Operator" + Graph *.Read.All. Government: -Environment USGov (GCC High) / USGovDoD (DoD); verify GCC High reporting parity where uncertain. > [CmdletBinding()] param([string]$OutDir = ".\APPASMT-EvidencePack")

How to run: Connect-MgGraph -Scopes "DeviceManagementApps.Read.All" GCC High: add -Environment USGov powershell -ExecutionPolicy Bypass -File .\APPASMT-Install-Health.ps1 -OutDir .\APPASMT-EvidencePack EXPECTED RESULTS Console summary plus: APPASMT-Install-Health.csv (per-app install success/failure/pending + failure rate) APPASMT-Version-Drift.csv (app titles with >1 live version) APPASMT-InstallHealth-Summary.json .NOTES Intune Solutions — application management assessment. READ-ONLY (reporting/analysis; no tenant writes). Least-privilege: Intune "Read Only Operator" + Graph *.Read.All. Government: -Environment USGov (GCC High) / USGovDoD (DoD); verify GCC High reporting parity where uncertain. > [CmdletBinding()] param([string]$OutDir = ".\APPASMT-EvidencePack")

APPASMT-Maturity-Evidence.ps1

Purpose: The aggregator for the "APPASMT - Maturity Model (9 Dimensions x 5 Levels)". It reads the four prior evidence files (estate, assignment, install-health, unmanaged) from the Evidence Pack, adds a few direct signals of its own (app-control / WDAC / AppLocker configuration profiles, co-management / SCCM workload signals, app configuration & protection policies), and emits a single scored data pack: for each of

the nine dimensions a SUGGESTED level (1-5) with the exact signal behind it. The suggested level is a starting point for the human assessor's APPASMT - Scoring Sheet, never the final word. READ-ONLY: Graph .Read.All scopes only; no tenant writes. HOW TO RUN Run the four collectors first so the Evidence Pack is populated, then: Connect-MgGraph -Scopes "DeviceManagementConfiguration.Read.All","DeviceManagementApps.Read.All","DeviceManagementRBAC.Read.All" GCC High: add -Environment USGov powershell -ExecutionPolicy Bypass -File .\APPASMT-Maturity-Evidence.ps1 -EvidenceDir .\APPASMT-EvidencePack EXPECTED RESULTS Console scorecard (9 dimensions, suggested level, signal) plus: APPASMT-EvidencePack.json (the machine-readable scored pack for the Scoring Sheet) APPASMT-Suggested-Scores.csv .NOTES Intune Solutions — application management assessment. READ-ONLY (aggregation/scoring; no tenant writes). Suggested levels are deterministic and transparent; the assessor confirms against the evidence tables in the Implementation Guide. Government: -Environment USGov (GCC High) / USGovDoD (DoD); verify GCC High parity where uncertain. > [CmdletBinding()] param([string]$EvidenceDir = ".\APPASMT-EvidencePack")

How to run: Run the four collectors first so the Evidence Pack is populated, then: Connect-MgGraph -Scopes "DeviceManagementConfiguration.Read.All","DeviceManagementApps.Read.All","DeviceManagementRBAC.Read.All" GCC High: add -Environment USGov powershell -ExecutionPolicy Bypass -File .\APPASMT-Maturity-Evidence.ps1 -EvidenceDir .\APPASMT-EvidencePack EXPECTED RESULTS Console scorecard (9 dimensions, suggested level, signal) plus: APPASMT-EvidencePack.json (the machine-readable scored pack for the Scoring Sheet) APPASMT-Suggested-Scores.csv .NOTES Intune Solutions — application management assessment. READ-ONLY (aggregation/scoring; no tenant writes). Suggested levels are deterministic and transparent; the assessor confirms against the evidence tables in the Implementation Guide. Government: -Environment USGov (GCC High) / USGovDoD (DoD); verify GCC High parity where uncertain. > [CmdletBinding()] param([string]$EvidenceDir = ".\APPASMT-EvidencePack")

APPASMT-Unmanaged-Detection.ps1

Purpose: Evidences dimensions D5 (Security & Application Control), D6 (Lifecycle & Retirement) and D9 (Migration Debt) by comparing the software actually DETECTED on managed devices (deviceManagement detectedApps) against the catalog of apps Intune deploys. Software present on devices but NOT deployed by Intune is manually installed / sideloaded — the shadow estate that app-control and lifecycle processes are supposed to govern. READ-ONLY: Graph .Read.All scopes only. HOW TO RUN Connect-MgGraph -Scopes "DeviceManagementApps.Read.All","DeviceManagementManagedDevices.Read.All" GCC High: add -Environment USGov powershell -ExecutionPolicy Bypass -File .\APPASMT-Unmanaged-Detection.ps1 -OutDir .\APPASMT-EvidencePack EXPECTED RESULTS Console summary plus: APPASMT-Unmanaged-Apps.csv (detected software not managed by Intune, with device counts) APPASMT-Unmanaged-Summary.json .NOTES Intune Solutions — application management assessment. READ-ONLY (detection/analysis; no tenant writes). Least-privilege: Intune "Read Only Operator" + Graph *.Read.All. Government: -Environment USGov (GCC High) / USGovDoD (DoD); verify GCC High detectedApps parity where uncertain. > [CmdletBinding()] param([string]$OutDir = ".\APPASMT-EvidencePack")

How to run: Connect-MgGraph -Scopes "DeviceManagementApps.Read.All","DeviceManagementManagedDevices.Read.All" GCC High: add -Environment USGov powershell -ExecutionPolicy Bypass -File .\APPASMT-Unmanaged-Detection.ps1 -OutDir .\APPASMT-EvidencePack EXPECTED RESULTS Console summary plus: APPASMT-Unmanaged-Apps.csv (detected software not managed by Intune, with device counts)

APPASMT-Unmanaged-Summary.json .NOTES Intune Solutions — application management assessment. READ-ONLY (detection/analysis; no tenant writes). Least-privilege: Intune "Read Only Operator" + Graph *.Read.All. Government: -Environment USGov (GCC High) / USGovDoD (DoD); verify GCC High detectedApps parity where uncertain. > [CmdletBinding()] param([string]$OutDir = ".\APPASMT-EvidencePack")